



# MCP Security Audit Report

AgentSign

---

projects

7 May 2026

Report ID: MCPS-MOUHZO50

# Executive Summary

WARN

Risk Score



0

CRITICAL

45

HIGH

23

MEDIUM

9

LOW

|             |                                                              |            |
|-------------|--------------------------------------------------------------|------------|
| SCAN SCOPE  | LANGUAGES                                                    | MCPS SDK   |
| 46 files    | python (31), json (8), javascript (5), <del>Not Script</del> | Not Script |
| 7,590 lines |                                                              |            |

# Risk Comparison: WITHOUT MCPS vs WITH MCPS

| CURRENT EXPOSURE |                                                                                                |   | WITH MCPS PROTECTION                                                                                                                                            |  |   |
|------------------|------------------------------------------------------------------------------------------------|---|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|--|---|
| MCP-01           | <b>Rug Pulls</b><br>Vulnerable pattern detected, no mitigation                                 | ● | <b>MITIGATED</b><br>Passport identity binding — tools are tied to verified agent identity<br><code>createPassport()</code> , <code>signPassport()</code>        |  | ● |
| MCP-03           | <b>Privilege Escalation via Tool Composition</b><br>No evidence of vulnerability or mitigation | ● | <b>MITIGATED</b><br>Tool integrity signing — each tool definition is cryptographically signed<br><code>signTool()</code> , <code>verifyTool()</code>            |  | ● |
| MCP-04           | <b>Cross-Server Request Forgery</b><br>Vulnerable pattern detected, no mitigation              | ● | <b>MITIGATED</b><br>Signed tool definitions prevent tampering during cross-server calls<br><code>signTool()</code> , <code>tool_hash</code>                     |  | ● |
| MCP-06           | <b>Indirect Prompt Injection via MCP</b><br>Vulnerable pattern detected, no mitigation         | ● | <b>MITIGATED</b><br>Signed message envelopes — all MCP messages carry cryptographic signatures<br><code>signMessage()</code> , <code>verifyMessage()</code>     |  | ● |
| MCP-07           | <b>Resource Exhaustion &amp; DoS</b><br>MCPS mitigation detected                               | ● | <b>MITIGATED</b><br>Passport verification middleware — all requests require valid signed identity<br><code>secureMCP()</code> , <code>verifyPassport()</code>   |  | ● |
| MCP-08           | <b>Insufficient Logging &amp; Audit</b><br>MCPS mitigation detected                            | ● | <b>MITIGATED</b><br>Signed audit trail — every message exchange is logged with cryptographic proof<br><code>auditLog()</code> , <code>onAudit()</code>          |  | ● |
| MCP-09           | <b>Insecure MCP-to-MCP Communication</b><br>MCPS mitigation detected                           | ● | <b>MITIGATED</b><br>Origin binding — passports encode allowed origins, verified on each request<br><code>validateOrigin()</code> , <code>passport.origin</code> |  | ● |
| MCP-10           | <b>Context Window Pollution</b><br>No evidence of vulnerability or mitigation                  | ● | <b>MITIGATED</b><br>Envelope isolation — signed message boundaries prevent context pollution<br><code>signMessage()</code> , <code>envelope</code>              |  | ● |

Risk Reduction

100 --> 100

# OWASP MCP Top 10 Compliance

Coverage: 3/8 risks mitigated by MCPS

| ID     | Risk                                      | Status           | Current State                              | With MCPS                                                                      |
|--------|-------------------------------------------|------------------|--------------------------------------------|--------------------------------------------------------------------------------|
| MCP-01 | Rug Pulls                                 | <div></div> FAIL | Vulnerable pattern detected, no mitigation | Passport identity binding — tools are tied to verified agent identity          |
| MCP-02 | Tool Poisoning                            | <div></div> N/A  | Not applicable to code-level analysis      | N/A — requires LLM-level defense                                               |
| MCP-03 | Privilege Escalation via Tool Composition | <div></div> WARN | No evidence of vulnerability or mitigation | Tool integrity signing — each tool definition is cryptographically signed      |
| MCP-04 | Cross-Server Request Forgery              | <div></div> FAIL | Vulnerable pattern detected, no mitigation | Signed tool definitions prevent tampering during cross-server calls            |
| MCP-05 | Sampling Manipulation                     | <div></div> N/A  | Not applicable to code-level analysis      | N/A — requires client-level defense                                            |
| MCP-06 | Indirect Prompt Injection via MCP         | <div></div> FAIL | Vulnerable pattern detected, no mitigation | Signed message envelopes — all MCP messages carry cryptographic signatures     |
| MCP-07 | Resource Exhaustion & DoS                 | <div></div> PASS | MCPS mitigation detected                   | Passport verification middleware — all requests require valid signed identity  |
| MCP-08 | Insufficient Logging & Audit              | <div></div> PASS | MCPS mitigation detected                   | Signed audit trail — every message exchange is logged with cryptographic proof |
| MCP-09 | Insecure MCP-to-MCP Communication         | <div></div> PASS | MCPS mitigation detected                   | Origin binding — passports encode allowed origins, verified on each request    |
| MCP-10 | Context Window Pollution                  | <div></div> WARN | No evidence of vulnerability or mitigation | Envelope isolation — signed message boundaries prevent context pollution       |

**Quick Fix**      `npm install mcp-secure`

Add MCPS to your MCP server to automatically mitigate the risks marked FAIL above.

# OWASP Agentic AI Top 10 Coverage

12 rules checked | 77 findings

| Rule   | OWASP  | MITRE     | STRIDE                 | Severity | Findings | Status      |
|--------|--------|-----------|------------------------|----------|----------|-------------|
| AS-001 | AA-03  | T1059     | Elevation of Privilege | CRITICAL | 0        | <div></div> |
| AS-002 | AA-05  | T1552     | Information Disclosure | HIGH     | 28       | <div></div> |
| AS-003 | AA-04  | T1078     | Elevation of Privilege | MEDIUM   | 5        | <div></div> |
| AS-004 | AA-02  | T1190     | Tampering              | HIGH     | 9        | <div></div> |
| AS-005 | AA-02  | T1055     | Tampering              | CRITICAL | 0        | <div></div> |
| AS-006 | AA-09  | T1610     | Elevation of Privilege | HIGH     | 0        | <div></div> |
| AS-007 | AA-06  | T1195     | Tampering              | LOW      | 9        | <div></div> |
| AS-008 | AA-01  | T1548     | Elevation of Privilege | HIGH     | 0        | <div></div> |
| AS-009 | AA-07  | T1059.007 | Tampering              | MEDIUM   | 0        | <div></div> |
| AS-010 | AA-08  | T1562.002 | Repudiation            | MEDIUM   | 18       | <div></div> |
| AS-011 | AA-10  | T1041     | Information Disclosure | HIGH     | 8        | <div></div> |
| AS-012 | MCP-07 | T1078.004 | Spoofing               | HIGH     | 0        | <div></div> |

# Detailed Findings

## #1 AS-010 -- LOGGING MONITORING

MEDIUM

OWASP: AA-08 | MITRE: T1562.002 | STRIDE: Repudiation

No evidence of logging, auditing, or monitoring in agent code

[0xwork/task\\_matcher.py:1](#)

**No logging, auditing, or monitoring detected**

```
> 1 | #!/usr/bin/env python3
  2 | """...
  3 | task_matcher.py - Lane C Enhancement
```

### Remediation:

1. Add structured logging (JSON) for every agent action, tool call, and decision. 2. Include agent\_id, timestamp, action, parameters, and outcome in each log entry. 3. Ship logs to a centralized SIEM (Splunk, Elastic, Sentinel). 4. Set up alerts for anomalous patterns (high-frequency tool calls, unusual commands). 5. Retain logs for compliance (90 days minimum).

## #2 AS-011 -- DATA EXFILTRATION

HIGH

OWASP: AA-10 | MITRE: T1041 | STRIDE: Information Disclosure

Dynamic HTTP requests that could exfiltrate sensitive data to attacker-controlled endpoints

[0xwork/task\\_matcher.py:146](#)

**Dynamic HTTP with sensitive data context**

```
144 | def fetch_tasks_api(min_bounty: float = 0, category: Optional[str] = None) -> List[Dict[str, Any]]:
145 |     """Fetch open tasks from 0xwork REST API."""
> 146 |     import urllib.request
147 |     import urllib.error
148 |
```

### Remediation:

1. Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains. 2. Use egress firewalls or network policies to restrict outbound traffic. 3. Redact sensitive data (tokens, credentials) before passing to agent context. 4. Log all outbound HTTP requests with destination and payload size. 5. Block requests containing sensitive patterns in the body.

## #3 AS-011 -- DATA EXFILTRATION

HIGH

OWASP: AA-10 | MITRE: T1041 | STRIDE: Information Disclosure

Dynamic HTTP requests that could exfiltrate sensitive data to attacker-controlled endpoints

[0xwork/task\\_matcher.py:147](#)

**Dynamic HTTP with sensitive data context**

```
145 |     """Fetch open tasks from 0xwork REST API."""
146 |     import urllib.request
> 147 |     import urllib.error
148 |
149 |     url = f"{OXWORK_API_BASE}/tasks?status=Open"
```

### Remediation:

1. Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains. 2. Use egress firewalls or network policies to restrict outbound traffic. 3. Redact sensitive data (tokens, credentials) before passing to agent context. 4. Log all outbound HTTP requests with destination and payload size. 5. Block requests containing sensitive patterns in the body.

#### #4 AS-011 -- DATA EXFILTRATION

HIGH

OWASP: AA-10 | MITRE: T1041 | STRIDE: Information Disclosure

Dynamic HTTP requests that could exfiltrate sensitive data to attacker-controlled endpoints

[0xwork/task\\_matcher.py:153](#)

**Dynamic HTTP with sensitive data context**

```
151 |         url += f"&category={category}"
152 |
> 153 |     req = urllib.request.Request(
154 |         url,...
155 |         headers={"Accept": "application/json", "User-Agent": "Manteclaw-TaskMatcher/1.0"},
```

##### Remediation:

1. Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains. 2. Use egress firewalls or network policies to restrict outbound traffic. 3. Redact sensitive data (tokens, credentials) before passing to agent context. 4. Log all outbound HTTP requests with destination and payload size. 5. Block requests containing sensitive patterns in the body.

#### #5 AS-011 -- DATA EXFILTRATION

HIGH

OWASP: AA-10 | MITRE: T1041 | STRIDE: Information Disclosure

Dynamic HTTP requests that could exfiltrate sensitive data to attacker-controlled endpoints

[0xwork/task\\_matcher.py:157](#)

**Dynamic HTTP with sensitive data context**

```
155 |         headers={"Accept": "application/json", "User-Agent": "Manteclaw-TaskMatcher/1.0"},
156 |     )
> 157 |     with urllib.request.urlopen(req, timeout=30) as resp:
158 |         data = json.loads(resp.read().decode("utf-8"))
159 |         tasks = data.get("tasks", data) if isinstance(data, dict) else data
```

##### Remediation:

1. Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains. 2. Use egress firewalls or network policies to restrict outbound traffic. 3. Redact sensitive data (tokens, credentials) before passing to agent context. 4. Log all outbound HTTP requests with destination and payload size. 5. Block requests containing sensitive patterns in the body.

#### #6 AS-003 -- EXCESSIVE PERMISSIONS

MEDIUM

OWASP: AA-04 | MITRE: T1078 | STRIDE: Elevation of Privilege

Agent requests high-risk permissions beyond minimum necessary

[arbitrage/arbitrage\\_bot.py:168](#)

**High-risk permission pattern: `parser.add_argument("--execute", action="store_true", help="Enable live trade ex`**

```
166 | async def main():
167 |     parser = argparse.ArgumentParser(description="Base L2 Reactive Arbitrage Bot")
> 168 |     parser.add_argument("--execute", action="store_true", help="Enable live trade execution")
169 |     parser.add_argument("--ws-url", default=None, help="WebSocket RPC URL (default: Alchemy wss)")
170 |     args = parser.parse_args()
```

##### Remediation:

1. Apply principle of least privilege — grant only permissions the agent needs for its specific task. 2. Use scoped API tokens (read-only where possible). 3. Implement permission escalation flow requiring human approval. 4. Audit permission usage and revoke unused grants.

#### #7 AS-003 -- EXCESSIVE PERMISSIONS

MEDIUM

OWASP: AA-04 | MITRE: T1078 | STRIDE: Elevation of Privilege

Agent requests high-risk permissions beyond minimum necessary

[arbitrage/arbitrage\\_bot.py:172](#)

**High-risk permission pattern: `dry_run = not args.execute`**

```
170 |     args = parser.parse_args()
171 |
> 172 |     dry_run = not args.execute
173 |
174 |     # Get Alchemy WebSocket URL
```

### Remediation:

1. Apply principle of least privilege — grant only permissions the agent needs for its specific task. 2. Use scoped API tokens (read-only where possible). 3. Implement permission escalation flow requiring human approval. 4. Audit permission usage and revoke unused grants.

## #8 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[arbitrage/price\\_monitor.py:121](#)

**Potential secret: possible hardcoded secret**

```
119 |         try:
120 |             pool_addr = factory.functions.getPool(token0_addr, token1_addr, fee).call()
> 121 |             if pool_addr == "0x0000000000000000000000000000000000000000":
122 |                 continue...
123 |
```

### Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

## #9 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[arbitrage/price\\_monitor.py:180](#)

**Potential secret: possible hardcoded secret**

```
178 |         pair_addr = factory.functions.getPair(token0_addr, token1_addr).call()
179 |
> 180 |         if pair_addr == "0x0000000000000000000000000000000000000000":
181 |             return None
182 |
```

### Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

## #10 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[botcoin/miner.py:14](#)

**Potential secret: possible hardcoded secret**

```
12 |         raise ValueError("BANKR_API_KEY environment variable required")
13 |     COORDINATOR_URL = os.environ.get("COORDINATOR_URL", "https://coordinator.agentmoney.net")
> 14 |     MINER_ADDRESS = "0x550c0cec65c9e585a0e59164f147a350e75a7a56"
15 |     TOKEN_ADDRESS = "0xA601877977340862Ca67f816eb079958E5bd0BA3"
16 |     STAKE_CONTRACT = "0xB2fbe0DB5A99B4E2Dd294dE64cEd82740b53A2Ea"
```

### Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.



## #11 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[botcoin/miner.py:15](#)

**Potential secret: possible hardcoded secret**

```
13 | COORDINATOR_URL = os.environ.get("COORDINATOR_URL", "https://coordinator.agentmoney.net")
14 | MINER_ADDRESS = "0x550c0cec65c9e585a0e59164f147a350e75a7a56"
> 15 | TOKEN_ADDRESS = "0xA601877977340862Ca67f816eb079958E5bd0BA3"
16 | STAKE_CONTRACT = "0xB2fbe0DB5A99B4E2Dd294dE64cEd82740b53A2Ea"...
17 | MIN_STAKE = 5_000_000 # whole tokens
```

### Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

## #12 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[botcoin/miner.py:16](#)

**Potential secret: possible hardcoded secret**

```
14 | MINER_ADDRESS = "0x550c0cec65c9e585a0e59164f147a350e75a7a56"
15 | TOKEN_ADDRESS = "0xA601877977340862Ca67f816eb079958E5bd0BA3"
> 16 | STAKE_CONTRACT = "0xB2fbe0DB5A99B4E2Dd294dE64cEd82740b53A2Ea"
17 | MIN_STAKE = 5_000_000 # whole tokens
18 |
```

### Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

## #13 AS-010 -- LOGGING MONITORING

MEDIUM

OWASP: AA-08 | MITRE: T1562.002 | STRIDE: Repudiation

No evidence of logging, auditing, or monitoring in agent code

[botcoin/miner.py:1](#)

**No logging, auditing, or monitoring detected**

```
> 1 | import os
2 | import sys
3 | import json
```

### Remediation:

1. Add structured logging (JSON) for every agent action, tool call, and decision. 2. Include agent\_id, timestamp, action, parameters, and outcome in each log entry. 3. Ship logs to a centralized SIEM (Splunk, Elastic, Sentinel). 4. Set up alerts for anomalous patterns (high-frequency tool calls, unusual commands). 5. Retain logs for compliance (90 days minimum).

## #14 AS-004 -- PROMPT INJECTION

HIGH

OWASP: AA-02 | MITRE: T1190 | STRIDE: Tampering

File input flows near prompt or instruction handling create injection vectors

[bounty/poc-path-traversal-aws-labs-mcp.py:7](#)

**File input near prompt/instruction handling — injection vector**

```
5 | Finding: mcp-open-path-traversal (CWE-22)
6 | File: src/aws-healthomics-mcp-server/aws-labs/aws-healthomics-mcp-server/utils/validation_utils.py
> 7 | Function: validate_readme_input() -> open(readme, 'r') without path sanitization
8 | MCP Tool: create_workflow() in workflow_management.py
9 |
```

### Remediation:

1. Sanitize all file contents before injecting into prompts (strip control chars, limit length). 2. Use structured input schemas — never concatenate raw file content into system prompts. 3. Implement input/output firewalls (Rebuff, LLM Guard). 4. Separate data plane from control plane in prompt construction.

## #15 AS-004 -- PROMPT INJECTION

HIGH

OWASP: AA-02 | MITRE: T1190 | STRIDE: Tampering

File input flows near prompt or instruction handling create injection vectors

[bounty/poc-path-traversal-awslabs-mcp.py:11](#)

**File input near prompt/instruction handling — injection vector**

```
9 |  
> 10 | The 'readme' parameter accepts a "local .md file path" and passes it directly  
11 | to open() without validating against a base directory. This allows arbitrary  
12 | file read via path traversal....  
13 |
```

### Remediation:

1. Sanitize all file contents before injecting into prompts (strip control chars, limit length). 2. Use structured input schemas — never concatenate raw file content into system prompts. 3. Implement input/output firewalls (Rebuff, LLM Guard). 4. Separate data plane from control plane in prompt construction.

## #16 AS-004 -- PROMPT INJECTION

HIGH

OWASP: AA-02 | MITRE: T1190 | STRIDE: Tampering

File input flows near prompt or instruction handling create injection vectors

[bounty/poc-path-traversal-awslabs-mcp.py:29](#)

**File input near prompt/instruction handling — injection vector**

```
27 |  
> 28 | When the HealthOmics MCP server processes this, validate_readme_input()  
29 | will detect it as a LOCAL_FILE (since it doesn't start with s3:// and  
30 | doesn't look like markdown content), then call open(readme, 'r') which  
31 | reads the target file.  
    | """
```

### Remediation:

1. Sanitize all file contents before injecting into prompts (strip control chars, limit length). 2. Use structured input schemas — never concatenate raw file content into system prompts. 3. Implement input/output firewalls (Rebuff, LLM Guard). 4. Separate data plane from control plane in prompt construction.

## #17 AS-004 -- PROMPT INJECTION

HIGH

OWASP: AA-02 | MITRE: T1190 | STRIDE: Tampering

File input flows near prompt or instruction handling create injection vectors

[bounty/poc-path-traversal-awslabs-mcp.py:52](#)

**File input near prompt/instruction handling — injection vector**

```
50 |  
> 51 | print(" 1. create_workflow() MCP tool accepts 'readme' parameter")  
52 | print(" 2. validate_readme_input(ctx, readme) detects LOCAL_FILE type")  
53 | print(" 3. open(readme, 'r') called WITHOUT path sanitization")  
54 | print()  
    | print("Proof-of-concept payloads:")
```

### Remediation:

1. Sanitize all file contents before injecting into prompts (strip control chars, limit length). 2. Use structured input schemas — never concatenate raw file content into system prompts. 3. Implement input/output firewalls (Rebuff, LLM Guard). 4. Separate data plane from control plane in prompt construction.

## #18 AS-004 -- PROMPT INJECTION

HIGH

OWASP: AA-02 | MITRE: T1190 | STRIDE: Tampering

File input flows near prompt or instruction handling create injection vectors

[bounty/poc-path-traversal-awslabs-mcp.py:96](#)

**File input near prompt/instruction handling — injection vector**

```
94 |             raise ValueError("Parent directory traversal not allowed")
95 |
> 96 |             return open(resolved, 'r', encoding='utf-8')
97 |         """...
98 |
```

### Remediation:

1. Sanitize all file contents before injecting into prompts (strip control chars, limit length). 2. Use structured input schemas — never concatenate raw file content into system prompts. 3. Implement input/output firewalls (Rebuff, LLM Guard). 4. Separate data plane from control plane in prompt construction.

## #19 AS-010 -- LOGGING MONITORING

MEDIUM

OWASP: AA-08 | MITRE: T1562.002 | STRIDE: Repudiation

No evidence of logging, auditing, or monitoring in agent code

[bounty/poc-path-traversal-awslabs-mcp.py:1](#)

**No logging, auditing, or monitoring detected**

```
> 1 | #!/usr/bin/env python3
2 | """
3 | PoC: Path Traversal in AWS HealthOmics MCP Server (awslabs/mcp)
```

### Remediation:

1. Add structured logging (JSON) for every agent action, tool call, and decision. 2. Include agent\_id, timestamp, action, parameters, and outcome in each log entry. 3. Ship logs to a centralized SIEM (Splunk, Elastic, Sentinel). 4. Set up alerts for anomalous patterns (high-frequency tool calls, unusual commands). 5. Retain logs for compliance (90 days minimum).

## #20 AS-004 -- PROMPT INJECTION

HIGH

OWASP: AA-02 | MITRE: T1190 | STRIDE: Tampering

File input flows near prompt or instruction handling create injection vectors

[bounty-hunter/bounty\\_scanner.py:355](#)

**File input near prompt/instruction handling — injection vector**

```
353 |             date_slug = datetime.utcnow().strftime("%Y%m%d")
354 |             report_file = REPORTS_DIR / f"mcp_audit_{safe_name}_{date_slug}.md"
> 355 |             with open(report_file, "w") as f:
356 |                 f.write(md)
357 |             result["report_file"] = str(report_file)
```

### Remediation:

1. Sanitize all file contents before injecting into prompts (strip control chars, limit length). 2. Use structured input schemas — never concatenate raw file content into system prompts. 3. Implement input/output firewalls (Rebuff, LLM Guard). 4. Separate data plane from control plane in prompt construction.

## #21 AS-004 -- PROMPT INJECTION

HIGH

OWASP: AA-02 | MITRE: T1190 | STRIDE: Tampering

File input flows near prompt or instruction handling create injection vectors

[bounty-hunter/bounty\\_scanner.py:362](#)

**File input near prompt/instruction handling — injection vector**

```
360 |             # Also save raw JSON
361 |             json_file = REPORTS_DIR / f"mcp_audit_{safe_name}"
_{datetime.utcnow().strftime('%Y%m%d')}.json"
> 362 |             with open(json_file, "w") as f:
363 |                 json.dump(result, f, indent=2)
364 |             result["json_file"] = str(json_file)
```

### Remediation:

1. Sanitize all file contents before injecting into prompts (strip control chars, limit length). 2. Use structured input schemas — never concatenate raw file content into system prompts. 3. Implement input/output firewalls (Rebuff, LLM Guard). 4. Separate data plane from control plane in prompt construction.

## #22 AS-004 -- PROMPT INJECTION

HIGH

OWASP: AA-02 | MITRE: T1190 | STRIDE: Tampering

File input flows near prompt or instruction handling create injection vectors

[bounty-hunter/bounty\\_scanner.py:379](#)

**File input near prompt/instruction handling — injection vector**

```
377 |         }
378 |         master_file = REPORTS_DIR / f"weekly_master_{datetime.utcnow().strftime('%Y%m%d')}.json"
> 379 |         with open(master_file, "w") as f:
380 |             json.dump(master, f, indent=2)...
381 |         print(f"[MASTER] Weekly summary: {master_file}")
```

### Remediation:

1. Sanitize all file contents before injecting into prompts (strip control chars, limit length). 2. Use structured input schemas — never concatenate raw file content into system prompts. 3. Implement input/output firewalls (Rebuff, LLM Guard). 4. Separate data plane from control plane in prompt construction.

## #23 AS-004 -- PROMPT INJECTION

HIGH

OWASP: AA-02 | MITRE: T1190 | STRIDE: Tampering

File input flows near prompt or instruction handling create injection vectors

[bounty-hunter/bounty\\_scanner.py:420](#)

**File input near prompt/instruction handling — injection vector**

```
418 |         targets = [args.target]
419 |         elif args.targets_file:
> 420 |             with open(args.targets_file) as f:
421 |                 targets = json.load(f)
422 |         else:
```

### Remediation:

1. Sanitize all file contents before injecting into prompts (strip control chars, limit length). 2. Use structured input schemas — never concatenate raw file content into system prompts. 3. Implement input/output firewalls (Rebuff, LLM Guard). 4. Separate data plane from control plane in prompt construction.

## #24 AS-010 -- LOGGING MONITORING

MEDIUM

OWASP: AA-08 | MITRE: T1562.002 | STRIDE: Repudiation

No evidence of logging, auditing, or monitoring in agent code

[clawbank/treasury.py:1](#)

**No logging, auditing, or monitoring detected**

```
> 1 | #!/usr/bin/env python3
2 | """
3 | CLAWBANK Treasury Management
```

### Remediation:

1. Add structured logging (JSON) for every agent action, tool call, and decision. 2. Include agent\_id, timestamp, action, parameters, and outcome in each log entry. 3. Ship logs to a centralized SIEM (Splunk, Elastic, Sentinel). 4. Set up alerts for anomalous patterns (high-frequency tool calls, unusual commands). 5. Retain logs for compliance (90 days minimum).

## #25 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[dune/dune\\_utils.py:26](#)

**Potential secret: possible hardcoded secret**

```
24 |  
25 | WALLETS = {  
> 26 |     "wallet_1_compromised": "0xC4Cf88b691D9b820040d861954d32e0C5f4538b7",  
27 |     "wallet_2_active": "0xD4E4b8e531d8AdAe126F400603361Ccda3931A8D", ...  
28 |     "wallet_3_bankr": "0x550c0cec65c9e585a0e59164f147a350e75a7a56",
```

### Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

## #26 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[dune/dune\\_utils.py:27](#)

**Potential secret: possible hardcoded secret**

```
25 | WALLETS = {  
26 |     "wallet_1_compromised": "0xC4Cf88b691D9b820040d861954d32e0C5f4538b7",  
> 27 |     "wallet_2_active": "0xD4E4b8e531d8AdAe126F400603361Ccda3931A8D",  
28 |     "wallet_3_bankr": "0x550c0cec65c9e585a0e59164f147a350e75a7a56",  
29 |     "wallet_4_fresh": "0xFC56950105883F46a3bB96ac9517A110724F2F27",
```

### Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

## #27 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[dune/dune\\_utils.py:28](#)

**Potential secret: possible hardcoded secret**

```
26 |     "wallet_1_compromised": "0xC4Cf88b691D9b820040d861954d32e0C5f4538b7",  
27 |     "wallet_2_active": "0xD4E4b8e531d8AdAe126F400603361Ccda3931A8D",  
> 28 |     "wallet_3_bankr": "0x550c0cec65c9e585a0e59164f147a350e75a7a56",  
29 |     "wallet_4_fresh": "0xFC56950105883F46a3bB96ac9517A110724F2F27",  
30 |     "registration_key": "0xE8663112EdaFaCaEf5711D49e42a11D37023FA32",
```

### Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

## #28 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[dune/dune\\_utils.py:29](#)

**Potential secret: possible hardcoded secret**

```
27 |     "wallet_2_active": "0xD4E4b8e531d8AdAe126F400603361Ccda3931A8D",  
28 |     "wallet_3_bankr": "0x550c0cec65c9e585a0e59164f147a350e75a7a56",  
> 29 |     "wallet_4_fresh": "0xFC56950105883F46a3bB96ac9517A110724F2F27",  
30 |     "registration_key": "0xE8663112EdaFaCaEf5711D49e42a11D37023FA32",  
31 |     "daydreams": "0xBE251af5140A0CEfe629364190e1840D27632aED",
```

### Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

## #29 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[dune/dune\\_utils.py:30](#)

**Potential secret: possible hardcoded secret**

```
28 |     "wallet_3_bankr": "0x550c0cec65c9e585a0e59164f147a350e75a7a56",
29 |     "wallet_4_fresh": "0xFC56950105883F46a3bB96ac9517A110724F2F27",
> 30 |     "registration_key": "0xE8663112EdaFaCaEf5711D49e42a11D37023FA32",
31 |     "daydreams": "0xBE251af5140A0CEfe629364190e1840D27632aED", ...
32 |     "zyfai_safe": "0x056f49F6F0De7A7d9154127aD0a419E8632Af239",
```

### Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

## #30 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[dune/dune\\_utils.py:31](#)

**Potential secret: possible hardcoded secret**

```
29 |     "wallet_4_fresh": "0xFC56950105883F46a3bB96ac9517A110724F2F27",
30 |     "registration_key": "0xE8663112EdaFaCaEf5711D49e42a11D37023FA32",
> 31 |     "daydreams": "0xBE251af5140A0CEfe629364190e1840D27632aED",
32 |     "zyfai_safe": "0x056f49F6F0De7A7d9154127aD0a419E8632Af239",
33 |     "nookplot_v2": "0xE8663112EdaFaCaEf5711D49e42a11D37023FA32",
```

### Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

## #31 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[dune/dune\\_utils.py:32](#)

**Potential secret: possible hardcoded secret**

```
30 |     "registration_key": "0xE8663112EdaFaCaEf5711D49e42a11D37023FA32",
31 |     "daydreams": "0xBE251af5140A0CEfe629364190e1840D27632aED",
> 32 |     "zyfai_safe": "0x056f49F6F0De7A7d9154127aD0a419E8632Af239",
33 |     "nookplot_v2": "0xE8663112EdaFaCaEf5711D49e42a11D37023FA32",
34 |     "workprotocol": "0xFC56950105883F46a3bB96ac9517A110724F2F27",
```

### Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

### #32 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[dune/dune\\_utils.py:33](#)

**Potential secret: possible hardcoded secret**

```
31 | "daydreams": "0xBE251af5140A0CEfe629364190e1840D27632aED",
32 | "zyfai_safe": "0x056f49F6F0De7A7d9154127aD0a419E8632Af239",
> 33 | "nookplot_v2": "0xE8663112EdaFaCaEf5711D49e42a11D37023FA32",
34 | "workprotocol": "0xFC56950105883F46a3bB96ac9517A110724F2F27",...
35 | "litcoin_claim": "0x550c0cec65c9e585a0e59164f147a350e75a7a56",
```

#### Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

### #33 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[dune/dune\\_utils.py:34](#)

**Potential secret: possible hardcoded secret**

```
32 | "zyfai_safe": "0x056f49F6F0De7A7d9154127aD0a419E8632Af239",
33 | "nookplot_v2": "0xE8663112EdaFaCaEf5711D49e42a11D37023FA32",
> 34 | "workprotocol": "0xFC56950105883F46a3bB96ac9517A110724F2F27",
35 | "litcoin_claim": "0x550c0cec65c9e585a0e59164f147a350e75a7a56",
36 | "0xwork": "0x550c0cec65c9e585a0e59164f147a350e75a7a56",
```

#### Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

### #34 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[dune/dune\\_utils.py:35](#)

**Potential secret: possible hardcoded secret**

```
33 | "nookplot_v2": "0xE8663112EdaFaCaEf5711D49e42a11D37023FA32",
34 | "workprotocol": "0xFC56950105883F46a3bB96ac9517A110724F2F27",
> 35 | "litcoin_claim": "0x550c0cec65c9e585a0e59164f147a350e75a7a56",
36 | "0xwork": "0x550c0cec65c9e585a0e59164f147a350e75a7a56",
37 | }
```

#### Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

### #35 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[dune/dune\\_utils.py:36](#)

**Potential secret: possible hardcoded secret**

```
34 | "workprotocol": "0xFC56950105883F46a3bB96ac9517A110724F2F27",
35 | "litcoin_claim": "0x550c0cec65c9e585a0e59164f147a350e75a7a56",
> 36 | "0xwork": "0x550c0cec65c9e585a0e59164f147a350e75a7a56",
37 | }
38 | }
```

### Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

## #36 AS-003 -- EXCESSIVE PERMISSIONS

MEDIUM

OWASP: AA-04 | MITRE: T1078 | STRIDE: Elevation of Privilege

Agent requests high-risk permissions beyond minimum necessary

[dune/dune\\_utils.py:48](#)

**High-risk permission pattern: url = f"{DUNE\_BASE\_URL}/query/{query\_id}/execute/status"**

```
46 | def get_query_status(query_id):
47 |     """Check execution status of a query"""
> 48 |     url = f"{DUNE_BASE_URL}/query/{query_id}/execute/status"
49 |     resp = requests.get(url, headers=DUNE_HEADERS)...
50 |     return resp.json()
```

### Remediation:

1. Apply principle of least privilege — grant only permissions the agent needs for its specific task. 2. Use scoped API tokens (read-only where possible). 3. Implement permission escalation flow requiring human approval. 4. Audit permission usage and revoke unused grants.

## #37 AS-003 -- EXCESSIVE PERMISSIONS

MEDIUM

OWASP: AA-04 | MITRE: T1078 | STRIDE: Elevation of Privilege

Agent requests high-risk permissions beyond minimum necessary

[dune/dune\\_utils.py:54](#)

**High-risk permission pattern: url = f"{DUNE\_BASE\_URL}/query/{query\_id}/execute"**

```
52 | def execute_query(query_id):
53 |     """Trigger fresh execution of a query"""
> 54 |     url = f"{DUNE_BASE_URL}/query/{query_id}/execute"
55 |     resp = requests.post(url, headers=DUNE_HEADERS)
56 |     return resp.json()
```

### Remediation:

1. Apply principle of least privilege — grant only permissions the agent needs for its specific task. 2. Use scoped API tokens (read-only where possible). 3. Implement permission escalation flow requiring human approval. 4. Audit permission usage and revoke unused grants.

## #38 AS-003 -- EXCESSIVE PERMISSIONS

MEDIUM

OWASP: AA-04 | MITRE: T1078 | STRIDE: Elevation of Privilege

Agent requests high-risk permissions beyond minimum necessary

[governance-bot/snapshot\\_client.py:25](#)

**High-risk permission pattern: """Execute a GraphQL query against the Snapshot hub."""**

```
23 |
24 |     def _graphql(self, query: str, variables: Optional[Dict] = None) -> Dict:
> 25 |         """Execute a GraphQL query against the Snapshot hub."""
26 |         payload = {"query": query}
27 |         if variables:
```

### Remediation:

1. Apply principle of least privilege — grant only permissions the agent needs for its specific task. 2. Use scoped API tokens (read-only where possible). 3. Implement permission escalation flow requiring human approval. 4. Audit permission usage and revoke unused grants.



## #39 AS-010 -- LOGGING MONITORING

MEDIUM

OWASP: AA-08 | MITRE: T1562.002 | STRIDE: Repudiation

No evidence of logging, auditing, or monitoring in agent code

[governance-bot/voting\\_engine.py:1](#)

**No logging, auditing, or monitoring detected**

```
> 1 | import json
  2 | import re...
  3 | from typing import Dict, List, Optional, Tuple
```

### Remediation:

1. Add structured logging (JSON) for every agent action, tool call, and decision. 2. Include agent\_id, timestamp, action, parameters, and outcome in each log entry. 3. Ship logs to a centralized SIEM (Splunk, Elastic, Sentinel). 4. Set up alerts for anomalous patterns (high-frequency tool calls, unusual commands). 5. Retain logs for compliance (90 days minimum).

## #40 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[litcoin/hybrid-miner.py:29](#)

**Potential secret: OpenAI/Stripe secret key**

```
27 | if not GROQ_KEY:
28 |     raise ValueError("GROQ_API_KEY environment variable required")
> 29 | CEREBRAS_KEY = os.environ.get("CEREBRAS_API_KEY",
"csk-5fyhth44rwwdx2cvjtpd3wfc8xcw2hmek388trrx4ydf4v5n")
30 | TOGETHER_KEY = os.environ.get("TOGETHER_API_KEY", "")
31 | SAMBANOVA_KEY = os.environ.get("SAMBANOVA_API_KEY", "b34dec5b-79ee-424a-80e4-715e0d6234d8")
```

### Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

## #41 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[litcoin/miner-v4.py:24](#)

**Potential secret: possible hardcoded secret**

```
22 | BANKR_KEY = os.getenv("BANKR_API_KEY", "")
23 | OPENROUTER_KEY = os.getenv("OPENROUTER_API_KEY", "")
> 24 | WALLET_ADDRESS = os.getenv("LITCOIN_WALLET_ADDRESS", "0xC4Cf88b691D9b820040d861954d32e0C5f4538b7")
25 |
26 | # === CONFIG ===
```

### Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

## #42 AS-010 -- LOGGING MONITORING

MEDIUM

OWASP: AA-08 | MITRE: T1562.002 | STRIDE: Repudiation

No evidence of logging, auditing, or monitoring in agent code

[litcoin/miner-v4.py:1](#)

**No logging, auditing, or monitoring detected**

```
> 1 | #!/usr/bin/env python3
  2 | """
  3 | LITCOIN Miner v4 - Official SDK Integration
```

### Remediation:

1. Add structured logging (JSON) for every agent action, tool call, and decision. 2. Include agent\_id, timestamp, action, parameters, and outcome in each log entry. 3. Ship logs to a centralized SIEM (Splunk, Elastic, Sentinel). 4. Set up alerts for anomalous patterns (high-frequency tool calls, unusual commands). 5. Retain logs for compliance (90 days minimum).

## #43 AS-010 -- LOGGING MONITORING

MEDIUM

OWASP: AA-08 | MITRE: T1562.002 | STRIDE: Repudiation

No evidence of logging, auditing, or monitoring in agent code

[litcoin/miner.py:1](#)

**No logging, auditing, or monitoring detected**

```
> 1 | #!/usr/bin/env python3
  2 | """
  3 | LITCOIN Mining Automation
```

### Remediation:

1. Add structured logging (JSON) for every agent action, tool call, and decision. 2. Include agent\_id, timestamp, action, parameters, and outcome in each log entry. 3. Ship logs to a centralized SIEM (Splunk, Elastic, Sentinel). 4. Set up alerts for anomalous patterns (high-frequency tool calls, unusual commands). 5. Retain logs for compliance (90 days minimum).

## #44 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[litcoin/standalone-miner/miner.py:12](#)

**Potential secret: possible hardcoded secret**

```
> 10 | source venv/bin/activate
  11 | export LITCOIN_SEED="your seed phrase"
  12 | export OPENROUTER_API_KEY="your key"
  13 | python3 standalone-miner.py
  14 | """
```

### Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

## #45 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[litcoin/standalone-miner/miner.py:33](#)

**Potential secret: possible hardcoded secret**

```
> 31 | COORDINATOR_URL = "https://api.litcoin.app"
  32 | OPENROUTER_URL = "https://openrouter.ai/api/v1"
  33 | WALLET_ADDRESS = "0xC4Cf88b691D9b820040d861954d32e0C5f4538b7"
  34 | MODEL = "inclusionai/ling-2.6-1t:free"
  35 |
```

### Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

## #46 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[litcoin/standalone-miner.py:12](#)

**Potential secret: possible hardcoded secret**

```
> 10 | source venv/bin/activate
  11 | export LITCOIN_SEED="your seed phrase"
  12 | export OPENROUTER_API_KEY="your key"
  13 | python3 standalone-miner.py
  14 | """
```

#### Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

### #47 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[bitcoin/standalone-miner.py:33](#)

**Potential secret: possible hardcoded secret**

```
31 | COORDINATOR_URL = "https://api.litcoin.app"
32 | OPENROUTER_URL = "https://openrouter.ai/api/v1"
> 33 | WALLET_ADDRESS = "0xC4Cf88b691D9b820040d861954d32e0C5f4538b7"
34 | MODEL = "inclusionai/ling-2.6-1t:free"
35 |
```

#### Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

### #48 AS-010 -- LOGGING MONITORING

MEDIUM

OWASP: AA-08 | MITRE: T1562.002 | STRIDE: Repudiation

No evidence of logging, auditing, or monitoring in agent code

[bitcoin/test-miner.py:1](#)

**No logging, auditing, or monitoring detected**

```
> 1 | #!/usr/bin/env python3
2 | """
3 | Litcoin Mining Test Script – Quick diagnostic + single task execution
```

#### Remediation:

1. Add structured logging (JSON) for every agent action, tool call, and decision. 2. Include agent\_id, timestamp, action, parameters, and outcome in each log entry. 3. Ship logs to a centralized SIEM (Splunk, Elastic, Sentinel). 4. Set up alerts for anomalous patterns (high-frequency tool calls, unusual commands). 5. Retain logs for compliance (90 days minimum).

### #49 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[mcp-base-automation/server.py:54](#)

**Potential secret: possible hardcoded secret**

```
52 |         return [TextContent(
53 |             type="text",
> 54 |             text="0x550c0cec65c9e585a0e59164f147a350e75a7a56"
55 |         )]
56 |     elif name == "get_agent_identity":
```

#### Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

## #50 AS-010 -- LOGGING MONITORING

MEDIUM

OWASP: AA-08 | MITRE: T1562.002 | STRIDE: Repudiation

No evidence of logging, auditing, or monitoring in agent code

[mcp-base-automation/server.py:1](#)

**No logging, auditing, or monitoring detected**

```
> 1 | #!/usr/bin/env python3
  2 | """
  3 | MCP Base Automation Server
```

### Remediation:

1. Add structured logging (JSON) for every agent action, tool call, and decision. 2. Include agent\_id, timestamp, action, parameters, and outcome in each log entry. 3. Ship logs to a centralized SIEM (Splunk, Elastic, Sentinel). 4. Set up alerts for anomalous patterns (high-frequency tool calls, unusual commands). 5. Retain logs for compliance (90 days minimum).

## #51 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[meshledger/register.js:9](#)

**Potential secret: possible hardcoded secret**

```
> 7 | name: 'Manteclaw-v2',
  8 | description: 'Base L2 automation agent – mining, research, and DeFi ops',
  9 | wallet_address: '0xFC56950105883F46a3bB96ac9517A110724F2F27',
 10 | chains: ['base'],
 11 | capabilities: ['python', 'solidity', 'automation', 'research', 'defi']
```

### Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

## #52 AS-007 -- SUPPLY CHAIN

LOW

OWASP: AA-06 | MITRE: T1195 | STRIDE: Tampering

External dependencies without integrity verification or lockfiles

[meshledger/register.js:1](#)

**Dependency without integrity verification**

```
> 1 | const { MeshLedger } = require('@meshledger/sdk');
  2 |
  3 | async function main() {
```

### Remediation:

1. Use lockfiles (package-lock.json, poetry.lock, requirements.txt with hashes). 2. Pin exact dependency versions — never use >= or latest. 3. Run npm audit / pip-audit in CI. 4. Use pip install --require-hashes for Python. 5. Verify package signatures where available.

## #53 AS-007 -- SUPPLY CHAIN

LOW

OWASP: AA-06 | MITRE: T1195 | STRIDE: Tampering

External dependencies without integrity verification or lockfiles

[meshledger/setup.js:1](#)

**Dependency without integrity verification**

```
> 1 | const { MeshLedger } = require('@meshledger/sdk');
  2 | require('dotenv').config({ path: '/root/.openclaw/workspace/projects/meshledger/.env' });
  3 |
```

### Remediation:

1. Use lockfiles (package-lock.json, poetry.lock, requirements.txt with hashes). 2. Pin exact dependency versions — never use >= or latest. 3. Run npm audit / pip-audit in CI. 4. Use pip install --require-hashes for Python. 5. Verify package signatures where available.

## #54 AS-007 -- SUPPLY CHAIN

LOW

OWASP: AA-06 | MITRE: T1195 | STRIDE: Tampering

External dependencies without integrity verification or lockfiles

[meshledger/setup.js:2](#)

**Dependency without integrity verification**

```
> 1 | const { MeshLedger } = require('@meshledger/sdk');
  2 | require('dotenv').config({ path: '/root/.openclaw/workspace/projects/meshledger/.env' });
  3 | ...
  4 | async function main() {
```

### Remediation:

1. Use lockfiles (package-lock.json, poetry.lock, requirements.txt with hashes). 2. Pin exact dependency versions — never use >= or latest. 3. Run npm audit / pip-audit in CI. 4. Use pip install --require-hashes for Python. 5. Verify package signatures where available.

## #55 AS-010 -- LOGGING MONITORING

MEDIUM

OWASP: AA-08 | MITRE: T1562.002 | STRIDE: Repudiation

No evidence of logging, auditing, or monitoring in agent code

[nookplot/agent.py:1](#)

**No logging, auditing, or monitoring detected**

```
> 1 | #!/usr/bin/env python3
  2 | """
  3 | NOOKPLOT Agent Coordinator
```

### Remediation:

1. Add structured logging (JSON) for every agent action, tool call, and decision. 2. Include agent\_id, timestamp, action, parameters, and outcome in each log entry. 3. Ship logs to a centralized SIEM (Splunk, Elastic, Sentinel). 4. Set up alerts for anomalous patterns (high-frequency tool calls, unusual commands). 5. Retain logs for compliance (90 days minimum).

## #56 AS-010 -- LOGGING MONITORING

MEDIUM

OWASP: AA-08 | MITRE: T1562.002 | STRIDE: Repudiation

No evidence of logging, auditing, or monitoring in agent code

[nookplot/check-guilds.py:1](#)

**No logging, auditing, or monitoring detected**

```
> 1 | import asyncio
  2 | import os
  3 | from nookplot_runtime import NookplotRuntime
```

### Remediation:

1. Add structured logging (JSON) for every agent action, tool call, and decision. 2. Include agent\_id, timestamp, action, parameters, and outcome in each log entry. 3. Ship logs to a centralized SIEM (Splunk, Elastic, Sentinel). 4. Set up alerts for anomalous patterns (high-frequency tool calls, unusual commands). 5. Retain logs for compliance (90 days minimum).

## #57 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[nookplot/create-guild.py:22](#)

**Potential secret: possible hardcoded secret**

```
> 20 |         name="Manteclaw Mining Collective",
  21 |         description="Autonomous agent mining and research guild on Base L2",
  22 |         members=[conn.address, "0xEA0aaD6DFa33D6EA4aec842C24D2015E3A4B3175"]
  23 |     )
  24 |     print(f"Create result: {result}")
```

### Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

## #58 AS-010 -- LOGGING MONITORING

MEDIUM

OWASP: AA-08 | MITRE: T1562.002 | STRIDE: Repudiation

No evidence of logging, auditing, or monitoring in agent code

[nookplot/create-guild.py:1](#)

**No logging, auditing, or monitoring detected**

```
> 1 | import asyncio
  2 | import os...
  3 | from nookplot_runtime import NookplotRuntime
```

### Remediation:

1. Add structured logging (JSON) for every agent action, tool call, and decision. 2. Include agent\_id, timestamp, action, parameters, and outcome in each log entry. 3. Ship logs to a centralized SIEM (Splunk, Elastic, Sentinel). 4. Set up alerts for anomalous patterns (high-frequency tool calls, unusual commands). 5. Retain logs for compliance (90 days minimum).

## #59 AS-010 -- LOGGING MONITORING

MEDIUM

OWASP: AA-08 | MITRE: T1562.002 | STRIDE: Repudiation

No evidence of logging, auditing, or monitoring in agent code

[nookplot/nookplot-agent.py:1](#)

**No logging, auditing, or monitoring detected**

```
> 1 | #!/usr/bin/env python3
  2 | """
  3 | Nookplot Agent – Quick reference and status checker.
```

### Remediation:

1. Add structured logging (JSON) for every agent action, tool call, and decision. 2. Include agent\_id, timestamp, action, parameters, and outcome in each log entry. 3. Ship logs to a centralized SIEM (Splunk, Elastic, Sentinel). 4. Set up alerts for anomalous patterns (high-frequency tool calls, unusual commands). 5. Retain logs for compliance (90 days minimum).

## #60 AS-010 -- LOGGING MONITORING

MEDIUM

OWASP: AA-08 | MITRE: T1562.002 | STRIDE: Repudiation

No evidence of logging, auditing, or monitoring in agent code

[nookplot/nookplot-full.py:1](#)

**No logging, auditing, or monitoring detected**

```
> 1 | #!/usr/bin/env python3
  2 | """
  3 | Nookplot Agent – Full setup and exploration.
```

### Remediation:

1. Add structured logging (JSON) for every agent action, tool call, and decision. 2. Include agent\_id, timestamp, action, parameters, and outcome in each log entry. 3. Ship logs to a centralized SIEM (Splunk, Elastic, Sentinel). 4. Set up alerts for anomalous patterns (high-frequency tool calls, unusual commands). 5. Retain logs for compliance (90 days minimum).

## #61 AS-010 -- LOGGING MONITORING

MEDIUM

OWASP: AA-08 | MITRE: T1562.002 | STRIDE: Repudiation

No evidence of logging, auditing, or monitoring in agent code

[nookplot/test-cliques.py:1](#)

**No logging, auditing, or monitoring detected**

```
> 1 | import asyncio
  2 | import os
  3 | from nookplot_runtime import NookplotRuntime
```

### Remediation:

1. Add structured logging (JSON) for every agent action, tool call, and decision. 2. Include agent\_id, timestamp, action, parameters, and outcome in each log entry. 3. Ship logs to a centralized SIEM (Splunk, Elastic, Sentinel). 4. Set up alerts for anomalous patterns (high-frequency tool calls, unusual commands). 5. Retain logs for compliance (90 days minimum).

## #62 AS-010 -- LOGGING MONITORING

MEDIUM

OWASP: AA-08 | MITRE: T1562.002 | STRIDE: Repudiation

No evidence of logging, auditing, or monitoring in agent code

[nookplot/test-cliques2.py:1](#)

**No logging, auditing, or monitoring detected**

```
> 1 | import asyncio
  2 | import os...
  3 | from nookplot_runtime import NookplotRuntime
```

### Remediation:

1. Add structured logging (JSON) for every agent action, tool call, and decision. 2. Include agent\_id, timestamp, action, parameters, and outcome in each log entry. 3. Ship logs to a centralized SIEM (Splunk, Elastic, Sentinel). 4. Set up alerts for anomalous patterns (high-frequency tool calls, unusual commands). 5. Retain logs for compliance (90 days minimum).

## #63 AS-010 -- LOGGING MONITORING

MEDIUM

OWASP: AA-08 | MITRE: T1562.002 | STRIDE: Repudiation

No evidence of logging, auditing, or monitoring in agent code

[nookplot/test-cliques3.py:1](#)

**No logging, auditing, or monitoring detected**

```
> 1 | import asyncio
  2 | import os
  3 | from nookplot_runtime import NookplotRuntime
```

### Remediation:

1. Add structured logging (JSON) for every agent action, tool call, and decision. 2. Include agent\_id, timestamp, action, parameters, and outcome in each log entry. 3. Ship logs to a centralized SIEM (Splunk, Elastic, Sentinel). 4. Set up alerts for anomalous patterns (high-frequency tool calls, unusual commands). 5. Retain logs for compliance (90 days minimum).

## #64 AS-010 -- LOGGING MONITORING

MEDIUM

OWASP: AA-08 | MITRE: T1562.002 | STRIDE: Repudiation

No evidence of logging, auditing, or monitoring in agent code

[nookplot/test-guilds.py:1](#)

**No logging, auditing, or monitoring detected**

```
> 1 | import asyncio
  2 | import os
  3 | from nookplot_runtime import NookplotRuntime
```

### Remediation:

1. Add structured logging (JSON) for every agent action, tool call, and decision. 2. Include agent\_id, timestamp, action, parameters, and outcome in each log entry. 3. Ship logs to a centralized SIEM (Splunk, Elastic, Sentinel). 4. Set up alerts for anomalous patterns (high-frequency tool calls, unusual commands). 5. Retain logs for compliance (90 days minimum).

## #65 AS-007 -- SUPPLY CHAIN

LOW

OWASP: AA-06 | MITRE: T1195 | STRIDE: Tampering

External dependencies without integrity verification or lockfiles

[x402-server/src/client.ts:7](#)

**Dependency without integrity verification**

```
> 5 | import { createPublicClient, http } from "viem";
  6 | import { base } from "viem/chains";
  7 | import dotenv from "dotenv";
  8 |
  9 | dotenv.config();
```

### Remediation:

1. Use lockfiles (package-lock.json, poetry.lock, requirements.txt with hashes). 2. Pin exact dependency versions — never use >= or latest. 3. Run npm audit / pip-audit in CI. 4. Use pip install --require-hashes for Python. 5. Verify package signatures where available.

## #66 AS-007 -- SUPPLY CHAIN

LOW

OWASP: AA-06 | MITRE: T1195 | STRIDE: Tampering

External dependencies without integrity verification or lockfiles

[x402-server/src/server.ts:2](#)

**Dependency without integrity verification**

```
> 1 | import express, { Request, Response } from "express";
  2 | import cors from "cors";
  3 | import dotenv from "dotenv";...
  4 | import { resolve, dirname } from "path";
```

### Remediation:

1. Use lockfiles (package-lock.json, poetry.lock, requirements.txt with hashes). 2. Pin exact dependency versions — never use >= or latest. 3. Run npm audit / pip-audit in CI. 4. Use pip install --require-hashes for Python. 5. Verify package signatures where available.

## #67 AS-007 -- SUPPLY CHAIN

LOW

OWASP: AA-06 | MITRE: T1195 | STRIDE: Tampering

External dependencies without integrity verification or lockfiles

[x402-server/src/server.ts:3](#)

**Dependency without integrity verification**

```
> 1 | import express, { Request, Response } from "express";
  2 | import cors from "cors";
  3 | import dotenv from "dotenv";
  4 | import { resolve, dirname } from "path";
  5 | import { fileURLToPath } from "url";
```

### Remediation:

1. Use lockfiles (package-lock.json, poetry.lock, requirements.txt with hashes). 2. Pin exact dependency versions — never use >= or latest. 3. Run npm audit / pip-audit in CI. 4. Use pip install --require-hashes for Python. 5. Verify package signatures where available.

## #68 AS-011 -- DATA EXFILTRATION

HIGH

OWASP: AA-10 | MITRE: T1041 | STRIDE: Information Disclosure

Dynamic HTTP requests that could exfiltrate sensitive data to attacker-controlled endpoints

[yield/yield\\_scanner.py:54](#)

**Dynamic HTTP with sensitive data context**

```
> 52 | def fetch_pools() -> List[Dict[str, Any]]:
  53 |     """Fetch all yield pools from DefiLlama."""
  54 |     import urllib.request
  55 |     import urllib.error
  56 |
```

### Remediation:

1. Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains. 2. Use egress firewalls or network policies to restrict outbound traffic. 3. Redact sensitive data (tokens, credentials) before passing to agent context. 4. Log all outbound HTTP requests with destination and payload size. 5. Block requests containing sensitive patterns in the body.

## #69 AS-011 -- DATA EXFILTRATION

HIGH

OWASP: AA-10 | MITRE: T1041 | STRIDE: Information Disclosure

Dynamic HTTP requests that could exfiltrate sensitive data to attacker-controlled endpoints

[yield/yield\\_scanner.py:55](#)

**Dynamic HTTP with sensitive data context**

```
> 53 |     """Fetch all yield pools from DefiLlama."""
  54 |     import urllib.request
  55 |     import urllib.error
  56 |
  57 |     req = urllib.request.Request(
```



### Remediation:

1. Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains. 2. Use egress firewalls or network policies to restrict outbound traffic. 3. Redact sensitive data (tokens, credentials) before passing to agent context. 4. Log all outbound HTTP requests with destination and payload size. 5. Block requests containing sensitive patterns in the body.

## #70 AS-011 -- DATA EXFILTRATION

HIGH

OWASP: AA-10 | MITRE: T1041 | STRIDE: Information Disclosure

Dynamic HTTP requests that could exfiltrate sensitive data to attacker-controlled endpoints

[yield/yield\\_scanner.py:57](#)

**Dynamic HTTP with sensitive data context**

```
55 | import urllib.error
56 |
> 57 | req = urllib.request.Request(
58 |     DEFILLAMA_POOLS_URL, ...
59 |     headers={"Accept": "application/json", "User-Agent": "Manteclaw-YieldScanner/1.0"},
```

### Remediation:

1. Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains. 2. Use egress firewalls or network policies to restrict outbound traffic. 3. Redact sensitive data (tokens, credentials) before passing to agent context. 4. Log all outbound HTTP requests with destination and payload size. 5. Block requests containing sensitive patterns in the body.

## #71 AS-011 -- DATA EXFILTRATION

HIGH

OWASP: AA-10 | MITRE: T1041 | STRIDE: Information Disclosure

Dynamic HTTP requests that could exfiltrate sensitive data to attacker-controlled endpoints

[yield/yield\\_scanner.py:61](#)

**Dynamic HTTP with sensitive data context**

```
59 |     headers={"Accept": "application/json", "User-Agent": "Manteclaw-YieldScanner/1.0"},
60 | )
> 61 | with urllib.request.urlopen(req, timeout=30) as resp:
62 |     data = json.loads(resp.read().decode("utf-8"))
63 |     # Defillama returns {data: [...]} or just [...]
```

### Remediation:

1. Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains. 2. Use egress firewalls or network policies to restrict outbound traffic. 3. Redact sensitive data (tokens, credentials) before passing to agent context. 4. Log all outbound HTTP requests with destination and payload size. 5. Block requests containing sensitive patterns in the body.

## #72 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[zyfai/check-zyfai.js:6](#)

**Potential secret: possible hardcoded secret**

```
4 | const apiKey = process.env.ZYFAI_API_KEY || "zyfai_your_api_key_here";
5 | const privateKey = process.env.PRIVATE_KEY || "0x" +
require("crypto").randomBytes(32).toString("hex");
> 6 | const userAddress = process.env.USER_ADDRESS || "0xC4cf88b691D9b820040d861954d32e0C5f4538b7";
7 | const chainId = 8453; // Base
8 |
```

### Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

## #73 AS-007 -- SUPPLY CHAIN

LOW

OWASP: AA-06 | MITRE: T1195 | STRIDE: Tampering

External dependencies without integrity verification or lockfiles

[zyfai/check-zyfai.js:1](#)

**Dependency without integrity verification**

```
> 1 | const { ZyfaiSDK, getSupportedChainIds } = require("@zyfai/sdk");
  2 | ...
  3 | async function main() {
```

### Remediation:

1. Use lockfiles (package-lock.json, poetry.lock, requirements.txt with hashes). 2. Pin exact dependency versions — never use >= or latest. 3. Run npm audit / pip-audit in CI. 4. Use pip install --require-hashes for Python. 5. Verify package signatures where available.

## #74 AS-007 -- SUPPLY CHAIN

LOW

OWASP: AA-06 | MITRE: T1195 | STRIDE: Tampering

External dependencies without integrity verification or lockfiles

[zyfai/check-zyfai.js:5](#)

**Dependency without integrity verification**

```
3 | async function main() {
4 |   const apiKey = process.env.ZYFAI_API_KEY || "zyfai_your_api_key_here";
> 5 |   const privateKey = process.env.PRIVATE_KEY || "0x" +
  require("crypto").randomBytes(32).toString("hex");
  6 |   const userAddress = process.env.USER_ADDRESS || "0xC4Cf88b691D9b820040d861954d32e0C5f4538b7";
  7 |   const chainId = 8453; // Base
```

### Remediation:

1. Use lockfiles (package-lock.json, poetry.lock, requirements.txt with hashes). 2. Pin exact dependency versions — never use >= or latest. 3. Run npm audit / pip-audit in CI. 4. Use pip install --require-hashes for Python. 5. Verify package signatures where available.

## #75 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[zyfai/discover.js:12](#)

**Potential secret: possible hardcoded secret**

```
10 |   }
11 |
> 12 |   const userAddress = "0xC4Cf88b691D9b820040d861954d32e0C5f4538b7";
  13 |   const chainId = 8453; // Base
  14 |
```

### Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

## #76 AS-007 -- SUPPLY CHAIN

LOW

OWASP: AA-06 | MITRE: T1195 | STRIDE: Tampering

External dependencies without integrity verification or lockfiles

[zyfai/discover.js:1](#)

**Dependency without integrity verification**

```
> 1 | const { ZyfaiSDK, getSupportedChainIds } = require("@zyfai/sdk");
  2 |
  3 | async function main() {
```

### Remediation:

1. Use lockfiles (package-lock.json, poetry.lock, requirements.txt with hashes). 2. Pin exact dependency versions — never use >= or latest. 3. Run npm audit / pip-audit in CI. 4. Use pip install --require-hashes for Python. 5. Verify package signatures where available.

## #77 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[zyfai/setup.js:8](#)

**Potential secret: possible hardcoded secret**

```
6 | const API_KEY = process.env.ZYFAI_API_KEY || "";
7 | const MNEMONIC = "state insane tooth rain scan march liberty man sick category noble divorce";
> 8 | const USER_ADDRESS = "0xC4Cf88b691D9b820040d861954d32e0C5f4538b7";
9 | const CHAIN_ID = 8453;...
10 |
```

### Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

# Remediation Checklist

Prioritized by severity. Address CRITICAL items first.

Quick Win

`npm install mcp-secure`

Adds cryptographic passports, signed messages, tool verification, and audit logging.

HIGH (3)

- ☐ **AS-011: data exfiltration**  
Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains.
- ☐ **AS-002: secret scan**  
Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv).
- ☐ **AS-004: prompt injection**  
Sanitize all file contents before injecting into prompts (strip control chars, limit length).

MEDIUM (2)

- ☐ **AS-010: logging monitoring**  
Add structured logging (JSON) for every agent action, tool call, and decision.
- ☐ **AS-003: excessive permissions**  
Apply principle of least privilege — grant only permissions the agent needs for its specific task.

LOW (1)

- ☐ **AS-007: supply chain**  
Use lockfiles (package-lock.json, poetry.lock, requirements.txt with hashes).

# Methodology & Standards

|               |                      |
|---------------|----------------------|
| Scanner       | mcps-audit v1.0.0    |
| Rules Version | 1.1.0                |
| Scanned At    | 07/05/2026, 04:16:29 |
| Report ID     | MCPS-MOUHZO50        |

# Referenced Standards

- OWASP MCP Top 10**  
Security risks for Model Context Protocol servers and clients.  
[owasp.org/www-project-mcp-top-10](https://owasp.org/www-project-mcp-top-10)
- OWASP Agentic AI Top 10**  
Security risks for autonomous AI agents.  
[owasp.org/www-project-agentic-ai-top-10](https://owasp.org/www-project-agentic-ai-top-10)
- MCPS -- IETF Internet-Draft**  
Cryptographic security layer for MCP: draft-sharif-mcps-secure-mcp.  
[datatracker.ietf.org/doc/draft-sharif-mcps-secure-mcp](https://datatracker.ietf.org/doc/draft-sharif-mcps-secure-mcp)
- MITRE ATT&CK**  
Adversary tactics and techniques knowledge base.  
[attack.mitre.org](https://attack.mitre.org)
- STRIDE Threat Model**  
Microsoft threat classification framework.  
[microsoft.com/en-us/security/blog/stride](https://microsoft.com/en-us/security/blog/stride)

**AgentSign**  
Cryptographic identity and security for AI agents  
[agentsign.dev](https://agentsign.dev)